

Case Study – Backend Engineer – Stephen Muema

Candidate	Stephen Muema
Role	Backend Engineer
Submission date	11 June 2026
Service	Reference Data Aggregation Service (RDAS)
GitHub repository	https://github.com/stephenmuema/loop-rdas-backend-engineer-assessment

1. Overview

RDAS is the single source of truth for country, currency, language and geographical reference data at LOOP DFS. It exposes clean REST/JSON APIs to all channels (mobile, web, partner APIs, internal ops) while consuming the third-party CountryInfo SOAP service internally. This removes the problems of every channel calling SOAP directly: inconsistent responses, poor performance, no filtering or pagination, no auditability, no shared caching, and SOAP credentials spread across applications.

The central design decision is a **materialized catalog**: RDAS builds and keeps a complete local copy of all countries and reference lists, and serves every read from it. A background harvester is the only component that calls SOAP.

REQUEST PATH (zero SOAP calls)

channels -> REST API -> Caffeine cache -> MySQL catalog

REFRESH PATH (the only SOAP, rate-limited + circuit-broken)

scheduler/startup -> Harvester -> CountryInfo SOAP service

This was implemented and verified end to end against the live SOAP service: a full harvest loaded **244 countries, 6 continents, 174 currencies and 407 languages** in about 2.5 minutes while staying under the 100 requests/minute quota; searches, filters, pagination, sorting, detail and currency-peer lookups all returned correct results from the catalog with no SOAP calls on the request path.

2. Part 1 - Solution design

A stateless Spring Boot service with three layers: a resilient SOAP client (the only SOAP caller), a harvester that builds the catalog by ISO-keyed upsert, and a read side that filters/paginates/sorts the catalog and caches hot results. The catalog lives in MySQL (H2 for local) and survives restarts, so the service serves even when SOAP is unavailable. Full write-up and diagram in `docs/architecture.md`.

3. Part 2 - API design

Method	Path	Purpose
GET	<code>/api/v1/countries</code>	Search by name; filter by continent/currency/language; paginate; sort.
GET	<code>/api/v1/countries/{isoCode}</code>	Full country detail.
GET	<code>/api/v1/countries/{isoCode}/currency-peers</code>	Countries sharing the currency.
GET	<code>/api/v1/continents</code> , <code>/currencies</code> , <code>/languages</code>	Reference lists.
GET	<code>/api/v1/admin/catalog/status</code>	Catalog freshness and counts.
POST	<code>/api/v1/admin/catalog/refresh</code>	Trigger a refresh (async, single-flight).

Responses use a stable paged envelope and a consistent error envelope. Full reference with request/response examples in `docs/api-documentation.md`.

4. Part 3 - Data processing (100 requests/minute limit)

- **Reducing SOAP traffic:** the read path makes zero SOAP calls; SOAP is used only by the scheduled harvester.
- **What is cached:** the durable MySQL catalog (all countries + reference lists) is the serving substrate; Caffeine caches hot read results in-process.
- **Expiration:** Caffeine entries expire after 30 minutes as a safety net; the catalog is refreshed, not aged out.
- **Refresh strategy:** a full harvest runs daily and on startup if empty/stale; it is paced (~120ms/call) and rate-limited to 90/min (under the 100 quota), and the read caches are evicted after each successful harvest.
- **Justification:** reference data changes very rarely, so a small bounded staleness buys a huge reduction in SOAP calls, sub-millisecond reads and independence from upstream availability.

5. Part 4 - Resilience (SOAP down for 6 hours)

- **Requests:** read APIs are unaffected - they serve from the catalog and never call SOAP. Users experience no failure.
- **Visibility:** `/admin/catalog/status` and the `catalog` health component expose `stale=true` once data ages; responses stay `200 OK`.
- **Fallbacks:** persisted catalog (serves even on a cold start during the outage); Resilience4j retry, timeouts, circuit breaker and fallbacks on the SOAP client; idempotent ISO-keyed upsert so retries are safe.
- **Monitoring/alerting:** Actuator health (catalog DOWN if empty, circuit breaker state) and Prometheus metrics; alert on breaker OPEN, `lastRefreshSuccessful=false`, and catalog age over threshold. Readiness is not tied to SOAP, so the outage never ejects a serving pod.

6. Part 5 - Implementation

Java 17, Spring Boot 3.2, Maven. Pagination and sorting via Spring Data `Pageable` (sortable fields whitelisted); filtering via JPA Specifications; caching via Spring Cache + Caffeine; global error handling via `@RestControllerAdvice` with correct status codes; input validation via Bean Validation. 49 tests (unit, `@DataJpaTest`, full web-layer integration) with a JaCoCo 80% gate, achieving about 97% line coverage. Containerised (multi-stage, non-root) with Docker Compose and Kubernetes manifests.

7. Part 6 - Engineering discussion

- **Limit cut to 10/min:** read path unchanged; only the harvester adapts - lower the rate limiter and make harvesting incremental/resumable across scheduled windows (ISO-keyed upsert already makes partial harvests safe). Slower refresh is acceptable for rarely-changing data.
- **20 million requests/day (~230 req/s avg):** all on the SOAP-free read path - scale stateless replicas behind the HPA, add a Redis cache tier and DB read replicas, edge-cache the near-static reference lists, and keep the harvester a leader-elected singleton so SOAP load stays constant.
- **One more week:** Redis distributed cache with warming, incremental/resumable harvest with change detection, OpenAPI/Swagger UI, auth on admin endpoints + an API gateway, Testcontainers and SOAP contract tests, distributed tracing, and Grafana dashboards/alerts.

8. Deliverables

Source code (GitHub), architecture diagram (`docs/architecture.md`), API documentation (`docs/api-documentation.md`), README, Kubernetes deployment scripts (`k8s/`), a Kubernetes deployment guide and a troubleshooting guide (`docs/`). A Postman collection is included under `postman/`.

Repository: <https://github.com/stephenmuema/loop-rdas-backend-engineer-assessment>